

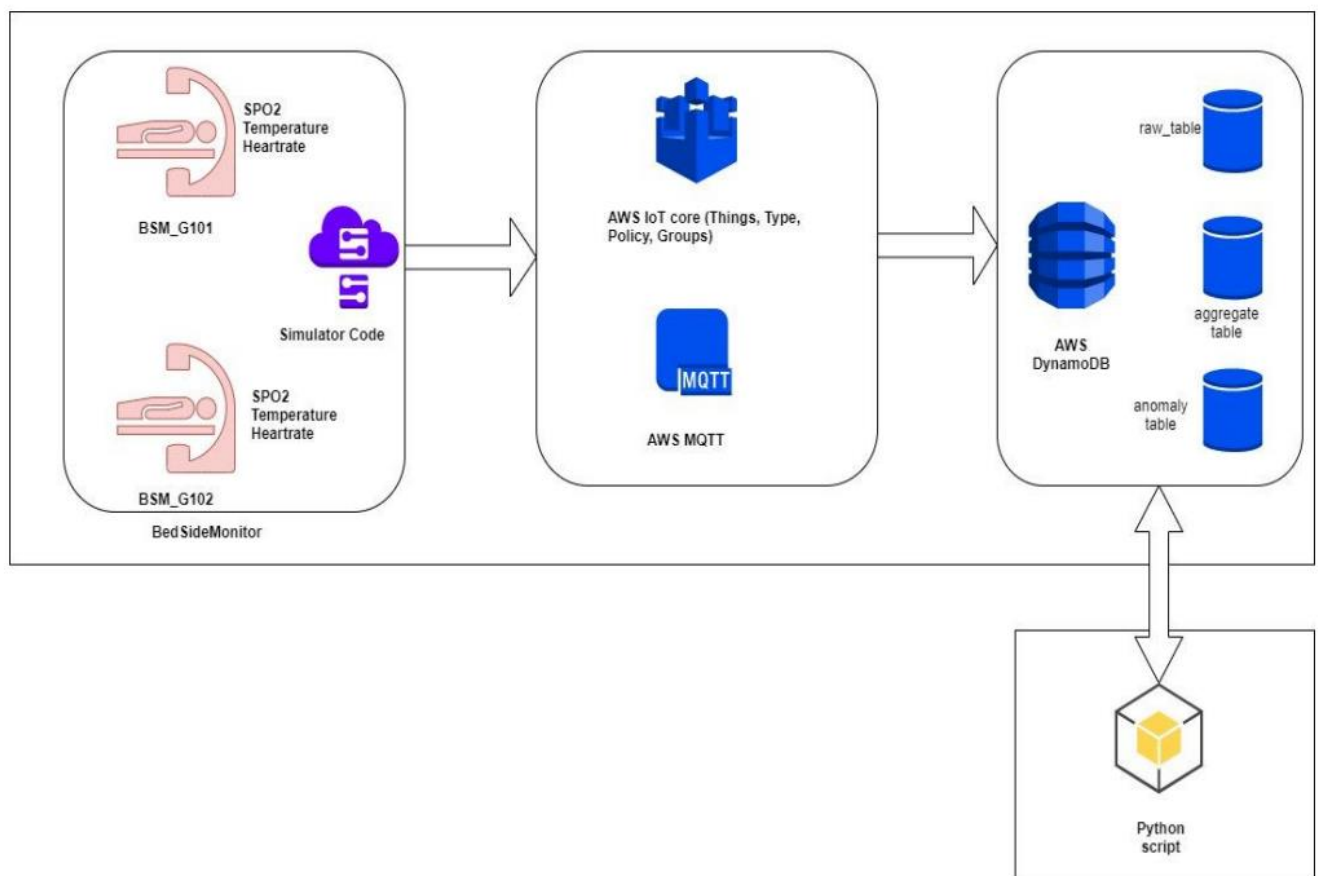
Hands on: Health Care with cloud processing

In this project, we'll be building up on the IoT Core and DynamoDB.

- The device simulator will publish Heart Rate, SpO2 (Oxygen Saturation) and Body Temperature values periodically.
- You'll need to channel that to the cloud (IoT Core → DynamoDB), and then aggregate data and detect anomalies.

Note:-

1. This is a simple example and may not follow some standard practices.
2. The focus is on the main flow, and not as much on error handling.
3. There will be some differences from standard flow since we are mimicking 'continuous monitoring by an always-on service' with a manually triggered program.



Program Organization

1. Device Simulator (BedSideMonitor.py)– This device simulator code will generate and push data to an IoT core endpoint.
 - a. The base code is given and similar to the device simulator used in practice session of AWS IoT Core.
 - b. Enhance code of BedsideMonitor.py to publish Heart Rate, SpO2 & Temperature periodically.
 - c. You can run multiple copies of this code to simulate multiple devices.
 - d. Run at least 2 instances of this code
2. In AWS - Create IoT Thing, Dynamo DB and push data into dynamo DB
 - a. Like the hands on done - Create IoT Things with Certificates, Policy
 - b. Create DynamoDB table (bsm_data)
 - c. Create IoT Core Rule to push data from Bedsidemonitor.py into this table
 - d. Run at least at least two instance of devices/things.
 - e. The data is being generated at 5 second frequencies for the three different sensor types.
3. Run device simulators for at least an hour continuously to generate enough data.
4. Aggregate the data in another table
 - a. Create a new table (bsm_agg_data) using code or manually via the UI.
 - b. Create another python code to model a database classes to access DynamoDB, similar to the previous hands on.
 - c. Provide functionality to aggregate the data from the three sensors individually, at the minute level. Calculate average, minimum and maximum values taking all values within a minute in account.
 - d. This should be stored programmatically in the aggregator table.

For example, when called with a time range of '2024-09-01 13:00:00 to 2024-09-01 14:00:00', it should create 60 rows per sensor type in the aggregated table, each being the aggregation of all values within a minute for that sensor data type. Entries can be skipped if there is no relevant data.

Amount of data going in Raw_data table if data is generated at 5 second interval –

In 1 minute – 12 records for each of 3 parameters = Total of 36 records

In 1 hour $36 * 60 = 2160$ record will be written in DB (720 record for each of 3 parameter)

Amount of data going in aggregate table

For 1 minute aggregated data of 1 minute for 3 parameter = 3 records

for 1 hour $3 * 60 = 180$ records (60 records for each parater)

5. Detect anomalies based on rules and store them
 - a. Create a rules config file using any format that you see fit, including json.
 - i. This should have the ability to define combination alert rules. One should be able to define an OR rule with min and max average values for any of the three data types. In addition, one should be able to define a trigger count so that the values should be breached continuously that many times, before an alert should be raised. For example, a rule can be {'type': 'HeartRate', 'avg_min': 55, 'avg_max': 105, 'trigger_count': 5} This should trigger an alert if

five continuous aggregated (at the minute level) average values are outside the min/max range. Please write an appropriate parser for these rules in the code. There should be at least two rules specified in the config file for demonstration purposes.

- b. Create a new table (bsm_alerts) using code or manually via the UI. Create the appropriate model to access it.
 - i. Provide functionality to run the rule check for a specified time range. It should check all rules on all devices individually within that time period and raise alerts as appropriate.
 - ii. These alerts should be printed on the console, and saved to the bsm_alerts table with an appropriate format, specifying deviceid, data type, the timestamp of the first instance of breach, and the rule that was breached.

Resources

1. Boto3: <https://boto3.amazonaws.com/v1/documentation/api/latest/guide/quickstart.html>
2. DynamoDB access from python:
<https://boto3.amazonaws.com/v1/documentation/api/latest/guide/dynamodb.html>
https://boto3.amazonaws.com/v1/documentation/api/latest/reference/services/dynamodb.html#DynamoDB.Client.list_tables
3. DynamoDB UI (Please watch the DynamoDB videos from the course):
<https://aws.amazon.com/getting-started/hands-on/create-nosql-table/>
4. IoT Core (Please review handson related to IoT Core):
<https://docs.aws.amazon.com/iot/latest/developerguide/what-is-aws-iot.html>